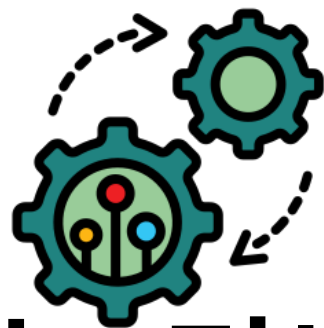




케이녹 리버싱 4회차 과제

17기 최동현 (202511688)

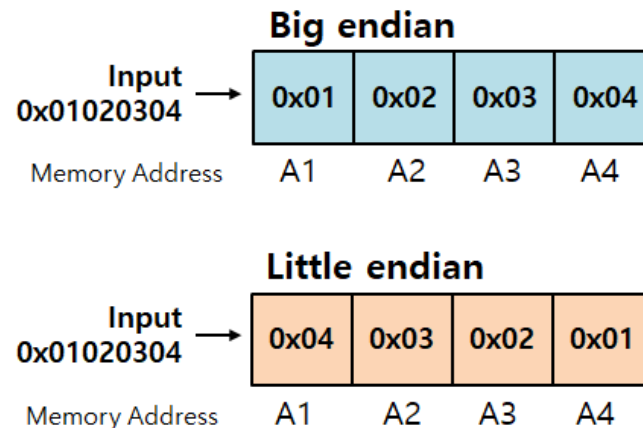
1. 엔디안 (Big vs Little)

• 엔디안 (Endianness)

： 컴퓨터의 메모리와 같은 1차원 공간에 다중 바이트(Multi-byte) 데이터를 저장할 때, 바이트를 배열하는 물리적 순서를 정의한 아키텍처 규약입니다.

* 빅엔디안 리틀엔디안 차이

- **빅엔디안 (Big Endian)** : 최상위 바이트(MSB, Most Significant Byte)를 메모리의 가장 낮은 주소에 먼저 저장하는 방식입니다. 사람이 숫자를 읽는 방식과 동일하며, TCP/IP 네트워크 프로토콜의 표준 바이트 오더(Network Byte Order)로 사용됩니다.
- **리틀엔디안 (Little Endian)** : 최하위 바이트(LSB, Least Significant Byte)를 메모리의 가장 낮은 주소에 먼저 저장하는 방식입니다. Intel 및 AMD의 **x86/x64 아키텍처**와 대부분의 ARM 프로세서가 채택하고 있는 방식입니다. CPU의 산술 논리 장치(ALU)가 하위 바이트부터 연산을 수행할 때 데이터 캐싱 및 연산 효율성이 높습니다.

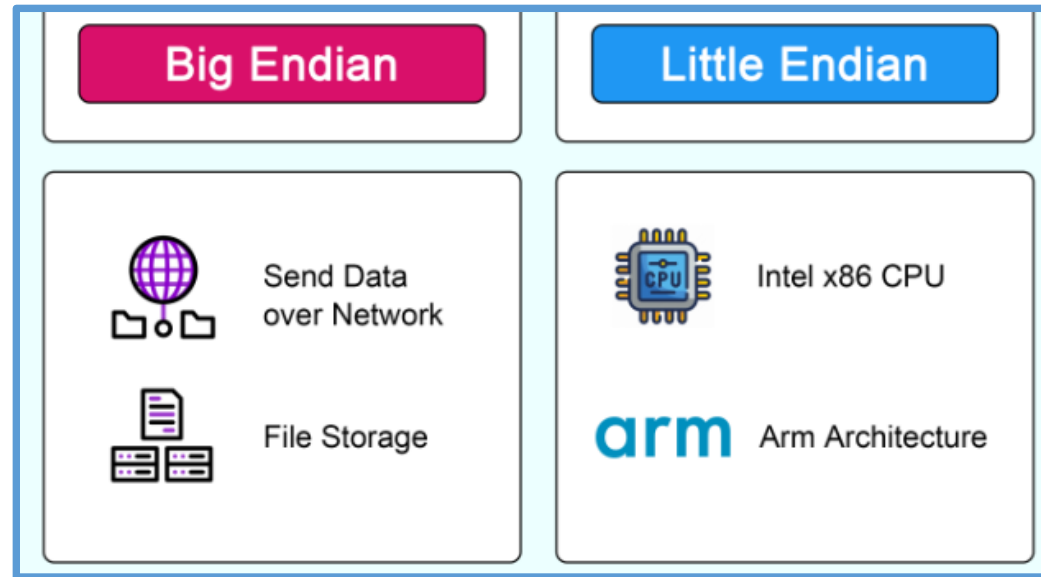


1. 엔디안 (Big vs Little)

• 엔디안마다 사용하는 CPU 아키텍처 비교

* 빅엔디안, 리틀엔디안

- **빅엔디안 (Big Endian)** : IBM z/Architecture, PowerPC (초기 모델) 등 주로 대형 서버나 네트워크 장비(라우터, FW, 스위치) 등에서 사용됩니다.
- **리틀엔디안 (Little Endian)** : Intel x86/AMD x64 등 현재 우리가 사용하는 대부분의 Desktop 혹은 Laptop PC 프로세서에서 사용됩니다.



2. 인코딩 (ASCII, UNICODE, UTF-16 ...)

• 인코딩 (Encoding)

： 문자 인코딩(Character Encoding)은 문자나 기호를 컴퓨터가 이해할 수 있는 이진 데이터(Bit/Byte)로 매핑하는 규격입니다.

* 인코딩 종류

- **ASCII** : 7비트(현대에는 1바이트)를 사용하여 영문 알파벳, 숫자, 제어 문자를 표현하는 가장 기본적인 인코딩 방식입니다. (A = 0x41, B = 0x42)
- **Unicode** : 전 세계의 모든 문자를 일관되게 표현할 수 있도록 설계된 국제 표준 문자 집합입니다. 이를 물리적 바이트로 인코딩하는 방식에 따라 UTF-8, UTF-16, UTF-32 등으로 나뉩니다.
- **Wide String (UTF-16 on Windows)** : Windows 운영체제의 내부 API는 기본적으로 UTF-16 Little Endian(와이드 스트링) 인코딩을 사용합니다. 이 방식은 영문 알파벳 하나를 표현할 때도 반드시 2바이트를 사용하므로, ASCII 'A'는 메모리 상에 0x41 0x00으로 저장됩니다. (C언어에서는 주로 wchar_t 타입을 사용하여 문자열을 만듦)

3. 메모리 구조 (코드, 데이터, 힙, 스택)

• 가상 메모리 구조

： 운영체제가 바이너리(PE/ELF)를 실행하여 프로세스를 생성할 때, 해당 프로세스에 할당하는 논리적인 가상 주소 공간(Virtual Address Space)의 분할 아키텍처입니다. 각 세그먼트(Segment)는 보안을 위해 서로 다른 읽기(R), 쓰기(W), 실행(X) 권한을 부여받습니다.

* 메모리 영역 종류

- **Code(text) 영역** : CPU가 실행할 기계어 명령어(Instruction)가 저장됩니다. 무결성을 유지하기 위해 **Read-Execute(R-X)** 권한만 부여되며 쓰기(Write)가 불가능합니다.
- **Data 영역** : 전역 변수(Global Variable)와 정적 변수(Static Variable)가 저장됩니다.
 - **.data 섹션** : 초기화된 변수 저장 (Read-Write)
 - **.bss 섹션** : 초기화되지 않은 변수 저장 (Read-Write, 실행 시 0으로 초기화됨)
- **Heap 영역** : 프로그램 실행 중 런타임(Runtime)에 동적으로 할당되는 데이터(C의 malloc, Python의 객체 등)가 저장됩니다. 낮은 주소에서 높은 주소 방향으로 성장하며 **Read-Write(RW-)** 권한을 가집니다.
- **Stack 영역** : 함수 호출 시 생성되는 지역 변수(Local Variable), 매개변수, 복귀 주소(Return Address)가 저장되는 임시 공간입니다. 높은 주소에서 낮은 주소 방향으로 성장하며 **Read-Write(RW-)** 권한을 가집니다.

4. 실습 과제 (StackFrame.exe)

00401020	55	push ebp	
00401021	8BEC	mov ebp,esp	
00401023	83EC 08	sub esp,8	
00401026	C745 FC 01000000	mov dword ptr ss:[ebp-4],1	[ebp-04]:BaseThreadInitThunk
0040102D	C745 F8 02000000	mov dword ptr ss:[ebp-8],2	
00401034	8B45 F8	mov eax,dword ptr ss:[ebp-8]	
00401037	50	push eax	
00401038	8B4D FC	mov ecx,dword ptr ss:[ebp-4]	[ebp-04]:BaseThreadInitThunk
0040103B	51	push ecx	ecx:"亮"
0040103C	E8 BFFFFFFF	call stackframe.401000	
00401041	83C4 08	add esp,8	
00401044	50	push eax	
00401045	68 84B34000	push stackframe.40B384	40B384: "%d\n"
0040104A	E8 18000000	call stackframe.401067	
0040104F	83C4 08	add esp,8	
00401052	33C0	xor eax,eax	
00401054	8BE5	mov esp,ebp	
00401056	5D	pop ebp	
00401057	C3	ret	

00401000	55	push ebp	
00401001	8BEC	mov ebp,esp	
00401003	83EC 08	sub esp,8	
00401006	8B45 08	mov eax,dword ptr ss:[ebp+8]	
00401009	8945 F8	mov dword ptr ss:[ebp-8],eax	
0040100C	8B4D 0C	mov ecx,dword ptr ss:[ebp+C]	
0040100F	894D FC	mov dword ptr ss:[ebp-4],ecx	[ebp-04]:BaseThreadInitThunk
00401012	8B45 F8	mov eax,dword ptr ss:[ebp-8]	
00401015	0345 FC	add eax,dword ptr ss:[ebp-4]	[ebp-04]:BaseThreadInitThunk
00401018	8BE5	mov esp,ebp	
0040101A	5D	pop ebp	
0040101B	C3	ret	

Main 함수 시작 주소 : 0x401020

Add 함수 시작 주소 : 0x401000 (이유 : 해당 함수 로직에 int 변수 2개 받음(8바이트) 그리고 파라미터 두 값을 받아와 eax,ecx에 가져와 스택영역에 넣고 eax에다가 add opcode로 더 해주는 것을 확인)

4. 실습 과제 (StackFrame.exe)

00401020	55	push ebp	
00401021	8BEC	mov ebp,esp	
00401023	83EC 08	sub esp,8	
00401026	C745 FC 01000000	mov dword ptr ss:[ebp-4],1	[ebp-04]:BaseThreadInitThunk
0040102D	C745 F8 02000000	mov dword ptr ss:[ebp-8],2	
00401034	8B45 F8	mov eax,dword ptr ss:[ebp-8]	
00401037	50	push eax	
00401038	8B4D FC	mov ecx,dword ptr ss:[ebp-4]	[ebp-04]:BaseThreadInitThunk
0040103B	51	push ecx	ecx:"亮"
0040103C	E8 BFFFFFFF	call stackframe.401000	
00401041	83C4 08	add esp,8	
00401044	50	push eax	
00401045	68 84B34000	push stackframe.40B384	40B384: "%d\n"
0040104A	E8 18000000	call stackframe.401067	
0040104F	83C4 08	add esp,8	
00401052	33C0	xor eax,eax	
00401054	8BE5	mov esp,ebp	
00401056	5D	pop ebp	
00401057	C3	ret	

Add 함수 복귀 주소: call stackframe.401000이 add함수 호출 명령어인데 이 밑의 명령어 주소가 return address 로 잡히기 때문에 add함수 첫부분이 push ebp로 시작하는 것을 알 수 있는데 그러면 ebp보다 높은 곳에 먼저 저장되니 ebp+4 영역에 해당 add 함수 복귀주소인 0x401041이 ebp+4에 담길것이다.

4. 실습 과제 (StackFrame.exe)

00401000	55	push ebp	
00401001	8BEC	mov ebp,esp	
00401003	83EC 08	sub esp,8	
00401006	8B45 08	mov eax,dword ptr ss:[ebp+8]	
00401009	8945 F8	mov dword ptr ss:[ebp-8],eax	
0040100C	8B4D 0C	mov ecx,dword ptr ss:[ebp+C]	
0040100F	894D FC	mov dword ptr ss:[ebp-4],ecx	[ebp-04]:BaseThreadInitThunk
00401012	8B45 F8	mov eax,dword ptr ss:[ebp-8]	
00401015	0345 FC	add eax,dword ptr ss:[ebp-4]	[ebp-04]:BaseThreadInitThunk
00401018	8BE5	mov esp,ebp	
0040101A	5D	pop ebp	
0040101B	C3	ret	

Add함수의 프레임 ebp+8과 ebp+C는 기본적으로 ebp위에 위치하는것을 알수있는데, 기본적으로 함수 호출할때 파라미터 값을 가져왔기 때문에 그 값이 있다 eax,ecx에 각각 1과 2를 담아서 호출했기때문에 ss:[ebp+8]하면 그 1과 2를 eax와 ecx에 저장할것이다.

Esp+4는 기본적으로 아까 return address가 ebp+4에 담긴다는것처럼 저기 mov ebp,esp에 보이다시피 스택 포인터의 top을 ebp에 기준점을 잡듯이 ebp+4와 esp+4는 같다고볼수있다. 그렇기때문에 복귀주소인 return address가 담길것이다. [ebp-4]와 [ebp-8]은 위에서 지역변수 int 두개만큼 8바이트 빼는것볼수있듯이 4바이트 지점 ebp-4에 저장할 스택공간이 있고 그다음 변수 ebp-8에 저장할 공간이 생긴다 왜냐하면 스택의 top을 ebp를 기준으로 했으니까.

4. 실습 과제 (StackFrame.exe)

00401020	55	push ebp	
00401021	8BEC	mov ebp,esp	
00401023	83EC 08	sub esp,8	
00401026	C745 FC 01000000	mov dword ptr ss:[ebp-4],1	[ebp-04]:BaseThreadInitThunk
0040102D	C745 F8 02000000	mov dword ptr ss:[ebp-8],2	
00401034	8B45 F8	mov eax,dword ptr ss:[ebp-8]	
00401037	50	push eax	
00401038	8B4D FC	mov ecx,dword ptr ss:[ebp-4]	[ebp-04]:BaseThreadInitThunk
0040103B	51	push ecx	ecx:"亮"
0040103C	E8 BFFFFFFF	call stackframe.401000	
00401041	83C4 08	add esp,8	
00401044	50	push eax	
00401045	68 84B34000	push stackframe.40B384	40B384: "%d\n"
0040104A	E8 18000000	call stackframe.401067	
0040104F	83C4 08	add esp,8	
00401052	33C0	xor eax,eax	
00401054	8BE5	mov esp,ebp	
00401056	5D	pop ebp	
00401057	C3	ret	

이 프로그램의 호출 규약: cdecl(C Declaration) C/C++ 호출 규약

그 근거: 함수 호출전에 굳이 ebp-4와 ebp-8에 값을 넣었음에도 불구하고 eax와 ecx에 따로 옮겨주는 모습을 볼 수 있는데, 이 후 호출 후 호출 당한 함수(Callee)가 아닌 호출한 함수에서 add esp, 8을 통해 매개변수 전달을 했던 영역을 직접 정리하고있음을 보여줍니다. 따라서 이는 cdecl 호출 방식입니다.



감사합니다

17기 최동현